МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра №806 «Вычислительная математика и программирование»

Курсовая работа по курсу
“Дискретный анализ”

Персистентные структуры данных

Студент: Дудовцев А.А.

Группа: М8О-315Б-23

Преподаватель: Сорокин С.А.

Оценка: _____

Дата: _____

Подпись: _____

Москва, 2026

Содержание

1. Репозиторий
2. Постановка задачи
3. Метод решения
4. Описание алгоритма
5. Описание программы
6. Тест производительности
7. Выводы

Репозиторий

https://github.com/AndreyDdvts/cp-DA

Постановка задачи

Вам дан набор горизонтальных отрезков, и набор точек. Для каждой точки определите сколько отрезков лежит строго над ней. Ваше решение должно работать online, то есть должно обрабатывать запросы по одному после построения необходимой структуры данных по входным данным. Чтение входных данных и запросов вместе и построение по ним общей структуры запрещено.

Формат ввода

В первой строке вам даны два числа n и m ($1 \le n, m \le 10^5$) — количество отрезков и количество точек соответственно. В следующих n строках вам заданы отрезки, в виде троек чисел l, r и h ($-10^9 \le l < r \le 10^9$, $-10^9 \le h \le 10^9$) — координаты x левой и правой границ отрезка и координата y отрезка соответственно. В следующих m строках вам даны пары чисел x, y ($-10^9 \le x, y \le 10^9$) — координаты точек.

Формат вывода

Для каждой точки выведите количество отрезков над ней.

Метод решения

Дан набор горизонтальных отрезков, каждый из которых задаётся числами l, r, h: l и r — границы отрезка по оси x, h — его высота. Для каждой точки (x, y) нужно посчитать количество отрезков, лежащих строго над ней, то есть отрезков, для которых $l \le x \le r$ и $h > y$. Из примера в условии видно, что концы отрезка включаются, а высота сравнивается строго. Сложность задачи в том, что точки поступают online, и ответ на каждую нужно выдать до чтения следующей.

Отрезков и точек может быть до 10^5, поэтому наивное решение, которое для каждой точки перебирает все отрезки, работает за $O(n \cdot m)$, то есть до 10^{10} операций, и не укладывается в ограничение по времени.

Зафиксируем координату x. Над вертикальной прямой, проходящей через x, лежат те отрезки, у которых $l \le x \le r$. Если для каждого x иметь массив A_x, где $A_x[h]$ — число таких отрезков на высоте h, то ответ на запрос (x, y) — это сумма элементов массива A_x на высотах больше y. Такую сумму на отрезке массива быстро считает дерево отрезков.

Дерево отрезков — это структура данных для работы с запросами на диапазонах. Оно позволяет за $O(\log n)$ выполнять следующие операции:

- Поиск суммы (или минимума, максимума, любой другой ассоциативной функции) элементов на отрезке массива
- Изменение значения элемента с индексом i

Дерево отрезков делит массив пополам, затем каждую половину снова пополам, и так до отдельных элементов. Каждая вершина дерева хранит сумму на своём подотрезке, а значение внутренней вершины равно сумме значений её детей. Высота дерева равна $O(\log n)$, поэтому и запрос, и изменение затрагивают $O(\log n)$ вершин.

Массив A_x меняется только в двух случаях: в точке $x = l$, где отрезок начинается, и в точке $x = r + 1$, где он заканчивается. Значит, различных

состояний массива не больше $2n + 1$, и каждое следующее отличается от предыдущего изменением одного элемента. Чтобы отвечать на запросы с любым x в любом порядке, нужно хранить все эти состояния одновременно. Для этого используется персистентное дерево отрезков.

Персистентное дерево отрезков — это дерево отрезков, которое сохраняет все свои предыдущие версии. При изменении одного элемента меняются только вершины на пути от корня до листа, а их $O(\log n)$. Вместо изменения этих вершин создаются их копии с новыми значениями, а остальные поддеревья новая версия берёт из старой без копирования. У каждой версии свой корень, и все старые корни остаются корректными. Поэтому каждое изменение стоит $O(\log n)$ времени и $O(\log n)$ дополнительной памяти.

Координаты по оси y могут быть очень большими ($-10^9 \le h \le 10^9$) и разреженными, поэтому строить дерево прямо по ним нельзя. Вместо этого используется сжатие координат: все различные высоты отрезков сортируются, и каждая заменяется своим номером от 0 до $K - 1$, где K — количество различных высот. Порядок высот при этом сохраняется, поэтому условие $h > y$ заменяется условием «номер высоты не меньше номера первой высоты, строго большей y».

Чтобы по координате точки x найти нужную версию дерева, версии хранятся в массиве, упорядоченном по координате x, и нужная версия находится бинарным поиском.

Описание алгоритма

Сначала выполняется сжатие высот: высоты всех отрезков записываются в массив heights, массив сортируется, и повторяющиеся значения удаляются. Номер высоты h — это её позиция в массиве heights, которая находится бинарным поиском.

Для каждого отрезка, заданного координатами l (левая граница), r (правая граница) и высотой h, мы создаём два события:

1. Событие начала отрезка — оно происходит в точке l. Это событие прибавляет 1 к количеству «активных» отрезков на высоте h.
2. Событие окончания отрезка — оно происходит в точке $r + 1$. Это событие вычитает 1 из количества «активных» отрезков на высоте h, так как в точке $r + 1$ отрезок уже не лежит над точками. Событие ставится в $r + 1$, а не в r, потому что правый конец отрезка включается, а все координаты целые.

Все $2n$ событий сортируются по координате x. Затем события обрабатываются слева направо, начиная с пустой версии дерева:

- Каждое событие создаёт новую версию дерева, в которой к элементу с номером высоты отрезка прибавлено +1 (начало отрезка) или −1 (конец отрезка).
- После обработки всех событий с одинаковой координатой x в массивы versionX и versionRoot записываются координата x и корень текущей версии дерева. Массив versionX получается отсортированным по возрастанию.

События с одинаковой координатой нужно обработать все вместе: если в одной точке один отрезок заканчивается, а другой начинается, то для запроса в этой точке нужна версия, в которой учтены оба события, а не промежуточная.

После этого начинается обработка точек. Каждая точка (x, y) читается, обрабатывается, и ответ выводится сразу, до чтения следующей точки:

1. С помощью upper_bound по массиву versionX находится последняя версия дерева, координата которой меньше или равна x. Если такой версии нет, точка лежит левее всех отрезков, и ответ равен 0.
2. С помощью upper_bound по массиву heights находится номер k первой высоты, строго большей y.
3. Ответ — сумма элементов с номерами от k до $K - 1$ в найденной версии дерева. Если $k = K$, то выше точки нет ни одной высоты, и ответ равен 0.

Алгоритм работает правильно: в версии, найденной для x, учтены все события с координатой не больше x. Отрезок дал вклад +1, если $l \le x$, и вклад −1, если $r + 1 \le x$. Суммарный вклад отрезка равен 1 только при $l \le x \le r$, то есть когда отрезок лежит над вертикалью x. Сумма по номерам высот от k до $K - 1$ учитывает ровно высоты $h > y$.

Описание программы

Программа состоит из следующих файлов: persistent_segment_tree.h — класс персистентного дерева отрезков, segments_above.h — класс, который строит версии дерева по отрезкам и отвечает на запросы, main.cpp — решение задачи, naive.cpp — наивное решение для проверки, bench.cpp — тест производительности.

Программа содержит структуру HorizontalSegment, которая хранит информацию о каждом отрезке: координаты левой и правой границы и высоту. Координаты хранятся в типе int64_t с запасом: при $r = 10^9$ координата события $r + 1 = 10^9 + 1$ ещё помещается в int, но 64-битный тип исключает переполнение при любых вычислениях с координатами.

В классе персистентного дерева отрезков PersistentSegmentTree все вершины всех версий хранятся в одном массиве std::vector. Каждая вершина хранит номера левого и правого ребёнка в этом массиве и сумму. Версия дерева задаётся номером своего корня. Вершина с номером 0 — общая пустая вершина: её сумма равна 0, а оба ребёнка ссылаются на неё же. Поэтому дерево не нужно строить заранее: пустая версия — это корень 0, а нужные вершины создаются при изменениях. Такое хранение занимает 12 байт на вершину и не требует отдельного выделения памяти на каждую вершину. Класс содержит методы:

- PersistentSegmentTree(int size, size_t expectedUpdates) — создание дерева для заданного количества элементов и резервирование памяти под вершины
- Root Add(Root version, int pos, int delta) — создание новой версии дерева, в которой к элементу pos прибавлено delta; возвращает корень новой версии, старая версия не меняется
- int64_t Sum(Root version, int l, int r) const — запрос суммы на отрезке [l, r] в версии version

Класс SegmentsAbove в конструкторе принимает массив отрезков, выполняет сжатие высот, создаёт и сортирует события и строит версии дерева. Он содержит метод:

- int64_t Query(int64_t x, int64_t y) const — количество отрезков, лежащих строго над точкой (x, y)

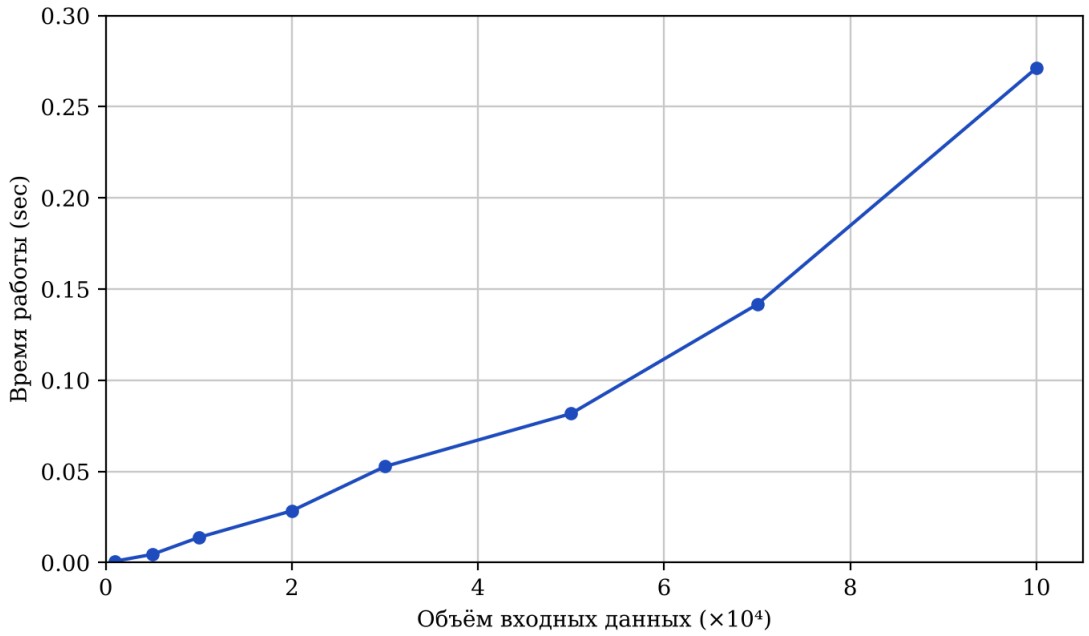
В функции main сначала читаются все отрезки и по ним строится объект SegmentsAbove. Затем точки читаются по одной, и для каждой сразу выводится ответ.

Для проверки корректности программа была запущена на примере из условия и на 3000 случайных тестов, ответы на которых сравнивались с ответами наивного решения naive.cpp. Все ответы совпали.

Тест производительности

Для измерения производительности создадим тесты с различными размерами входных данных. Подсчёт времени будем производить с помощью библиотеки chrono, которая позволяет фиксировать время в начале и конце работы. Для каждого теста будем задавать размеры n и m: (1000, 1000), (5000, 5000), (10000, 10000), (20000, 20000), (30000, 30000), (50000, 50000), (70000, 70000) и (100000, 100000), и генерировать n случайных отрезков и m случайных точек с координатами от -10^9 до 10^9. Время для каждого размера усреднено по 5 запускам.

На графике представлена зависимость времени построения структуры и подсчёта количества отрезков над точками от объёма входных данных.



Для сравнения, наивное решение на тесте (100000, 100000) работает около 47 секунд, а персистентное дерево отрезков — 0,27 секунды, то есть примерно в 170 раз быстрее. Вся программа вместе с вводом и выводом на максимальном тесте работает 0,34 секунды (ответ на каждую точку выводится сразу, до чтения следующей) и использует 53 МБ памяти при ограничениях 5 секунд и 256 МБ.

Сжатие высот и сортировка событий выполняются за $O(n \log n)$. Каждое изменение дерева выполняется за $O(\log n)$, и так как для каждого отрезка происходит два изменения (в начале и в конце), построение всех версий занимает $O(n \log n)$ времени. Каждое изменение создаёт $O(\log n)$ новых вершин, поэтому дерево занимает $O(n \log n)$ памяти.

Для каждой точки выполняются два бинарных поиска и запрос суммы, то есть $O(\log n)$. Таким образом, общее время работы алгоритма: $O((n + m) \log n)$.

Выводы

В ходе данной курсовой работы были изучены персистентные структуры данных, которые позволяют хранить все предыдущие версии структуры и обращаться к любой из них, не тратя на это лишнего времени.

В частности, было реализовано персистентное дерево отрезков, с помощью которого решена задача о количестве горизонтальных отрезков, лежащих строго над точкой. Двумерная задача свелась к последовательности одномерных: при движении слева направо по оси x получаются версии дерева, по одной на каждую координату начала или конца отрезка, и для ответа на запрос достаточно найти нужную версию и посчитать сумму. Благодаря персистентности запросы обрабатываются online, в том порядке, в котором приходят, без повторной обработки всех данных. Также было использовано сжатие координат по оси y, чтобы размер дерева зависел от количества отрезков, а не от диапазона координат.

При реализации нужно было учесть граничные случаи: включение концов отрезка, строгое сравнение высот, обработку событий с одинаковой координатой и событие окончания отрезка в точке $r + 1$. Хранение вершин дерева в одном массиве вместо отдельных объектов уменьшило расход памяти и ускорило работу программы.